UNIFAAT 2026.2

09 - React: Referências e Efeitos

Prof. Luan Lourenço

Tudo dentro do componente roda de novo

PAI · PARÂMETRO

`<Tarefa nome={…} />`

"Estudar React"

"Ler o briefing"

FUNÇÃO TAREFA() · CÓDIGO

```
function Tarefa({ nome }: Props) {
    const [isDone, setIsDone] = useState(false)

    function aoClicar() {
        setIsDone(!isDone)
    }

    return (
        <li onClick={aoClicar}>
            <input type="checkbox"
                checked={isDone} readOnly />
            {nome}
        </li>
    )
}
```

FRAME · TELA

☐ Estudar React

renderizações: **1**

1 · clique no <li>: o navegador chama `onClick`

2 · `aoClicar()` chama `setIsDone`: o React guarda o estado novo

3 · `Tarefa()` roda de novo, inteira

primeira renderização

Variáveis, `const` derivados, funções internas e o JSX: **tudo** é recriado a cada renderização. Só o estado (e a ref) vive fora da função.

Parâmetros desestruturados: uma foto por renderização

```
# o componente recebe UM objeto: props
function TaskItem(props: TaskItemProps) {
  const { task, onToggle } = props
# o mesmo, desestruturando na assinatura
function TaskItem({ task, onToggle }: TaskItemProps)
```

```
# valor padrão · renomear · aninhar
function Paginacao({
  page = 1,
  user: { id, name: userName }
}: PaginacaoProps)
```

- → `task` e `onToggle` são `const` **novas a cada execução**: a foto dos parâmetros daquela renderização
- → Parâmetro é **somente leitura**: quem muda o dado é o dono do estado, avisado por callback
- → O pai renderizou? O filho roda de novo, **mesmo com parâmetros iguais**

A renderização cai de cima para baixo

1. Clique num usuário: `handleSelect` chama `setSelected(user)`
2. **UsersApp** (o dono do estado) roda **inteira**, com o `selected` novo
3. O JSX dela contém `<UserTasks user={selected} />`: o filho roda em seguida, com o `user` novo
4. O React compara o JSX novo com o anterior e altera **só o que mudou** no DOM
5. Se o **filho** tem estado e ele muda, só o filho (e quem está abaixo) roda: o **pai não**

```
# a árvore
Page
  └ UsersApp ← estado: selected
    └ UserTasks ← user (parâmetro)
# para baixo: parâmetros
# para cima: callbacks
```

A tipagem mantém pai e filho coerentes

```
# o contrato do filho
type TaskItemProps = {
  task: Task
  onToggle: (task: Task) => void
  variant?: "success" | "danger"
}
<TaskItem task={task} /> ← ERRO: falta onToggle
```

```
# estado que começa vazio
const [selected, setSelected] =
  useState<User | null>(null)
if (selected === null) return ...
selected.name ← aqui já é User
# genérico: T = o que a API devolve
useAxios<PaginatedResponse<Task>>(url)
```

- O erro aparece **antes de rodar**: parâmetro faltando, união fora da lista, callback com outra assinatura
- Depois de um *early return*, o TypeScript **estreita** o tipo: `null` some

`useRef`: o mesmo objeto do heap em toda renderização

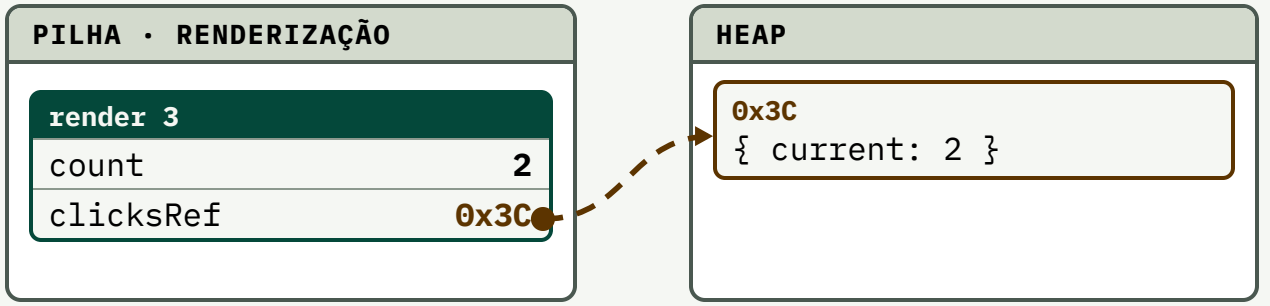
CÓDIGO

```
const [count, setCount] = useState(0)
const clicksRef = useRef(0)

// na ref: altera o objeto, sem renderizar
clicksRef.current += 1

// no estado: o React renderiza de novo
setCount(atual => atual + 1)
```

MEMÓRIA DO PROGRAMA



Observação: o `count` é uma **foto nova** a cada execução (frame novo). A ref é **um só objeto**: o endereço `0x3C` não muda, só o conteúdo.

Variável, estado ou referência?

	variável local	useState	useRef
onde vive	frame da pilha	guardado pelo React	**objeto** `{ current }` **no heap**
sobrevive às renderizações	não	sim (foto nova)	**sim (mesmo objeto)**
mudar renderiza	não	**sim**	**não**
como mudar	atribuição	`setEstado(...)`	`ref.current = ...`
serve para	cálculo temporário	o que **aparece** na tela	**o que só o código lembra**

regra O usuário precisa **ver** a mudança? **Estado**. Só o código precisa lembrar? **Ref**. E nunca leia nem escreva `ref.current` durante a renderização: só em **handlers** e **efeitos**.

Ref para o HTML e para componentes

CLIQUE NUMA PARTE

```tsx
const inputRef = useRef<HTMLInputElement>(null)   ① criar
<input ref={inputRef} />
function aoFocar() {
    inputRef.current?.focus()
}
// também: click() · select() · scrollIntoView()
function TextField({ ref }: TextFieldProps) {
    return <input ref={ref} />
}
```

① Criar a ref

Criar. O generic diz **qual elemento** vai morar ali. Começa em `null`: antes de montar não há elemento.

Ref guarda o que o código lembra, não a tela

```
# campo não controlado
const name =
  nameRef.current?.value
await taskCreateApi(
  idUser, { name })
o valor vai direto ao POST;
nenhuma renderização por tecla
```

```
# último dado enviado
if (key ===
  lastSearchRef.current)
  return
lastSearchRef.current = key
mesma busca: não chama
o backend de novo
```

```
# id do timer (debounce)
window.clearTimeout(
  timerRef.current)
timerRef.current =
  window.setTimeout(buscar, 400)
cancela o timer anterior
sem renderizar
```

Três usos, um motivo: o dado **não aparece na tela**, então guardá-lo em estado só renderizaria à toa.

Efeito: a ação da transição de estados

Máquina de Turing	React
estado atual	`useState` e parâmetros
função de transição	o componente: estado + parâmetros → tela
entrada	evento do usuário, resposta da API
ação ao transitar	**`useEffect`**

> **ideia** Depois que o **estado é atualizado** (e a tela já foi desenhada), executa-se o **efeito dessa mudança**: buscar na API, assinar um evento, mexer no título da aba.

O componente só calcula tela. O efeito é o lugar do que acontece **fora** dela.

Renderiza, desenha a tela, e só então o efeito

1. O evento chama `setEstado`: o React agenda uma renderização
2. A **função do componente roda** inteira e devolve o JSX
3. O React atualiza o DOM e o navegador **pinta a tela**
4. Só então rodam os **efeitos** cujas dependências mudaram (a **limpeza** do anterior vem antes)
5. O efeito chama `setEstado` com a resposta da API: **volta ao passo 1**

```
useEffect(() => {
  // 1 · o efeito: DEPOIS da tela
  return () => {
    // 2 · a limpeza
  }
}, [deps]) // 3 · quando roda de novo
```

Dependências: primitivo por valor, objeto por endereço

deps	o efeito roda
`[]`	uma vez, ao montar
`[user.id]`	ao montar e quando `user.id` muda
omitido	depois de **toda** renderização

```
# ERRADO · loop infinito
const params = { page: 1 } ← endereço novo
    a cada renderização
useEffect(..., [params])
# certo: dependa de primitivos
useEffect(..., [user.id, page])
```

- O React compara cada dependência com `Object.is`: é a mesma regra do **heap** da Aula 08
- Efeito muda estado, estado renderiza, objeto novo: o efeito dispara de novo. E de novo

`AbortController`: cancela a requisição velha

1. Clica em **Ana**: o efeito dispara `GET /users/7/tasks` com o `signal` do controller dela
2. Antes da resposta, clica em **Bruno**: a dependência `user.id` muda
3. **ERRADO, sem abort:** a resposta de Ana chega depois e **sobrescreve** a tela de Bruno
4. **Com abort:** a limpeza chama `controller.abort()`; o GET de Ana rejeita com `CanceledError` e o `isCancel` ignora
5. Só a resposta de **Bruno** vira estado: a tela e o nome combinam

```
# ① cria o controle (API nativa do navegador)
const controller = new AbortController()
# ② entrega o signal à requisição
get(url, { signal: controller.signal })
# ③ na limpeza do efeito, cancela
return () => controller.abort()
# É do navegador, não do React nem do Axios.
# O abort só faz o cliente parar de esperar:
# o que o servidor já processou não desfaz.
```

O hook feito por nós: useAxios e a tela dele

```
export function useAxios<T>(url, onEvent?) {
  const [data, setData] = useState<T | null>(null)
  const [error, setError] = useState("")
  const [isLoading, setIsLoading] = useState(url !== null)
  const onEventRef = useRef(onEvent) // ref: fora das deps
  useEffect(() => { onEventRef.current = onEvent })
  useEffect(() => {
    const controller = new AbortController()
    const request = { isPending: true }
    async function load() { /* GET + estados */ }
    load() // o efeito em si não é async
    return () => {
      if (!request.isPending) return // já chegou
      controller.abort()
    }
  }, [url, attempt]) // só primitivos
  return { data, error, isLoading, refetch }
}
```

/hooks HookLab · View do projeto

isLoading **false** error **""** data **3 task(s)**

trocar 3x rápido: o que o hook contou

```
14:02:11 GET começou  /users/7/tasks
14:02:11 GET CANCELADO /users/7/tasks
14:02:11 GET começou  /users/9/tasks
14:02:11 GET CANCELADO /users/9/tasks
14:02:11 GET começou  /users/12/tasks
14:02:12 GET chegou   /users/12/tasks
```

Os estados **nascem no hook** e o componente **herda**. O `onEvent` mora numa **ref**, e o `abort` só roda se a requisição ainda está no ar (`isPending`).

`useAxios`: o componente herda o estado do hook

USERTASKS · CÓDIGO

```
const { data, error, isLoading } =
    useAxios<PaginatedResponse<Task>>(
        `/users/${user.id}/tasks`)

if (isLoading)
    return <p>Carregando...</p>

return <ul>{ tasks }</ul>
```

ESTADOS HERDADOS DO HOOK

UserTasks()

data	**{ data: [...] }**
error	**""**
isLoading	**false**

setData + setIsLoading(false):
1 renderização

TELA

Tasks de Ana
Estudar React · feita
Ler o briefing · pendente

Cada chamada ao hook tem os **seus** estados: o hook reaproveita **lógica**, não dados.

Qual ferramenta para qual problema

preciso de...	uso
um valor que **aparece na tela** e muda	`useState`
um valor **calculado** de estado e parâmetros	conta solta no componente
acessar o HTML, `focus()`, dado de bastidor sem renderizar	`useRef`
algo **depois da tela**, por causa de uma mudança de estado	`useEffect`
reaproveitar estado e efeito entre componentes	`use...` (custom hook)
reagir a uma **ação do usuário**	handler (`onClick`, `onSubmit`)

---

estado → renderização → tela → **efeito** → estado. A ref é a gaveta que **não** entra nesse ciclo.

TF 09

React: Referências e Efeitos

enunciado O TF é o **README do diretório da aula** no repositório do professor:
`https://github.com/luan-tavares/unifaat-2026-frontend/tree/main/aulas/09`

- **O que pede:** app React + Vite + TypeScript com **Axios** na API pública JSONPlaceholder; **custom hook** `useAxios` **próprio** com `AbortController` na limpeza; `useRef` para foco e dado de bastidor; `useEffect` no `document.title`
- **Entrega:** branch `RA-tf-09` + diretório com o mesmo nome + `tf.md` (Nome, RA, GitHub) + **Pull Request** para o `main` do professor, título `TF-09 · RA · Nome`
- **Prazo:** a definir